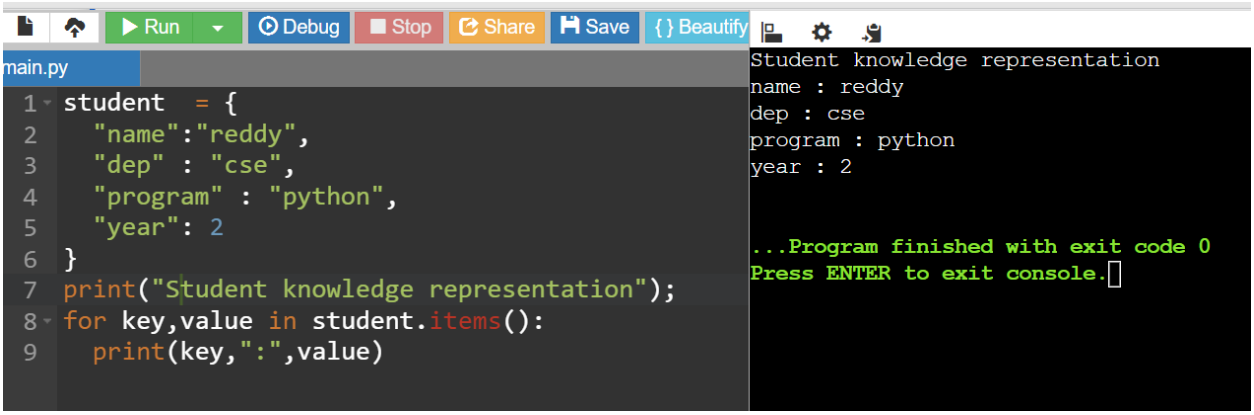
DSA1604-Knowledge engineering and data science

Lab execution part-1

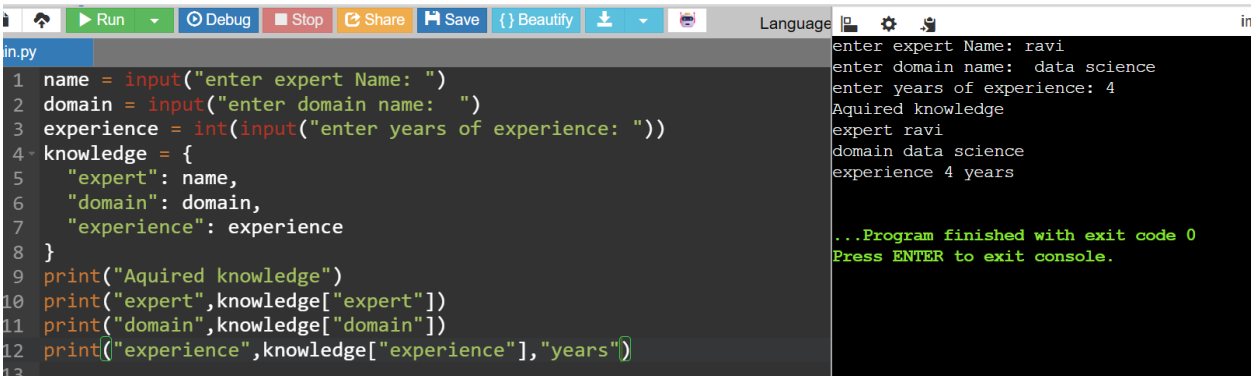
Name : P.Sudharshan Reddy

Reg.No : 192372204

Experiment 1: Knowledge Representation Using Python



Experiment 2: Knowledge Acquisition



Experiment 3: Knowledge-Based Course Recommendation System

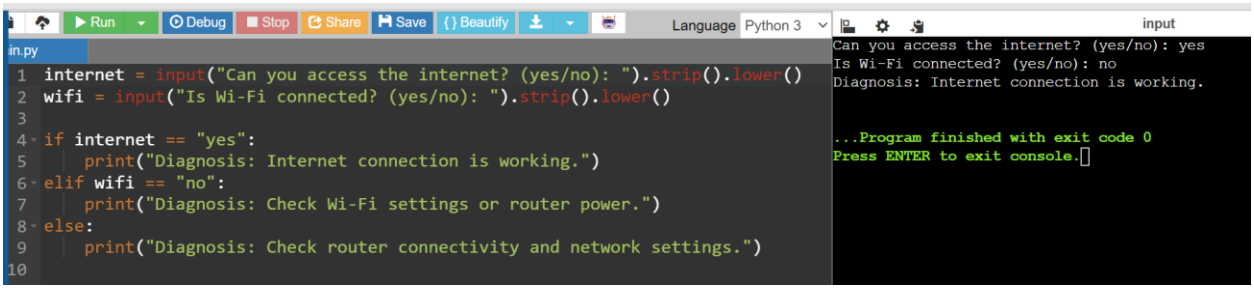


```
interest = input("Enter your interest (AI, Data, Web): ").strip().lower()
if interest == "ai":
    course = "Artificial Intelligence"
elif interest == "data":
    course = "Data Science"
elif interest == "web":
    course = "Web Development"
else:
    course = "No matching course found"
print("Recommended Course:", course)
```

```
Enter your interest (AI, Data, Web): ai
Recommended Course: Artificial Intelligence

...Program finished with exit code 0
Press ENTER to exit console.
```

Experiment 4: Rule-Based Expert System for Network Troubleshooting



```
internet = input("Can you access the internet? (yes/no): ").strip().lower()
wifi = input("Is Wi-Fi connected? (yes/no): ").strip().lower()

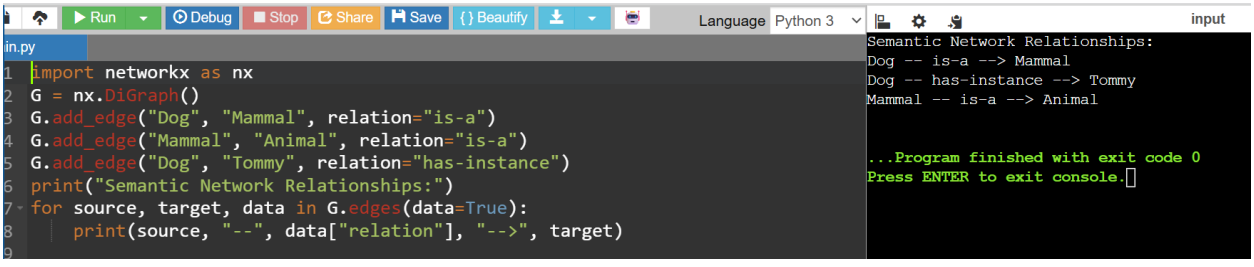
if internet == "yes":
    print("Diagnosis: Internet connection is working.")
elif wifi == "no":
    print("Diagnosis: Check Wi-Fi settings or router power.")
else:
    print("Diagnosis: Check router connectivity and network settings.")

```

```
Can you access the internet? (yes/no): yes
Is Wi-Fi connected? (yes/no): no
Diagnosis: Internet connection is working.

...Program finished with exit code 0
Press ENTER to exit console.
```

Experiment 5: Semantic Network Using NetworkX



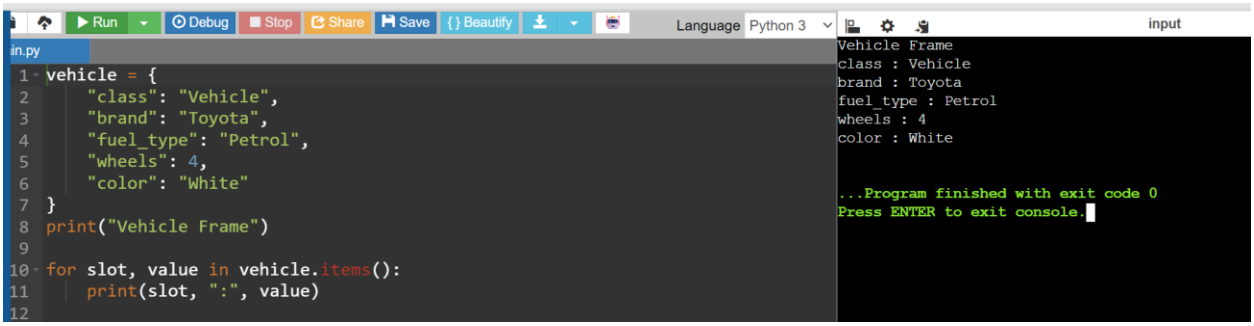
```
import networkx as nx
G = nx.DiGraph()
G.add_edge("Dog", "Mammal", relation="is-a")
G.add_edge("Mammal", "Animal", relation="is-a")
G.add_edge("Dog", "Tommy", relation="has-instance")
print("Semantic Network Relationships:")
for source, target, data in G.edges(data=True):
    print(source, "--", data["relation"], "-->", target)

```

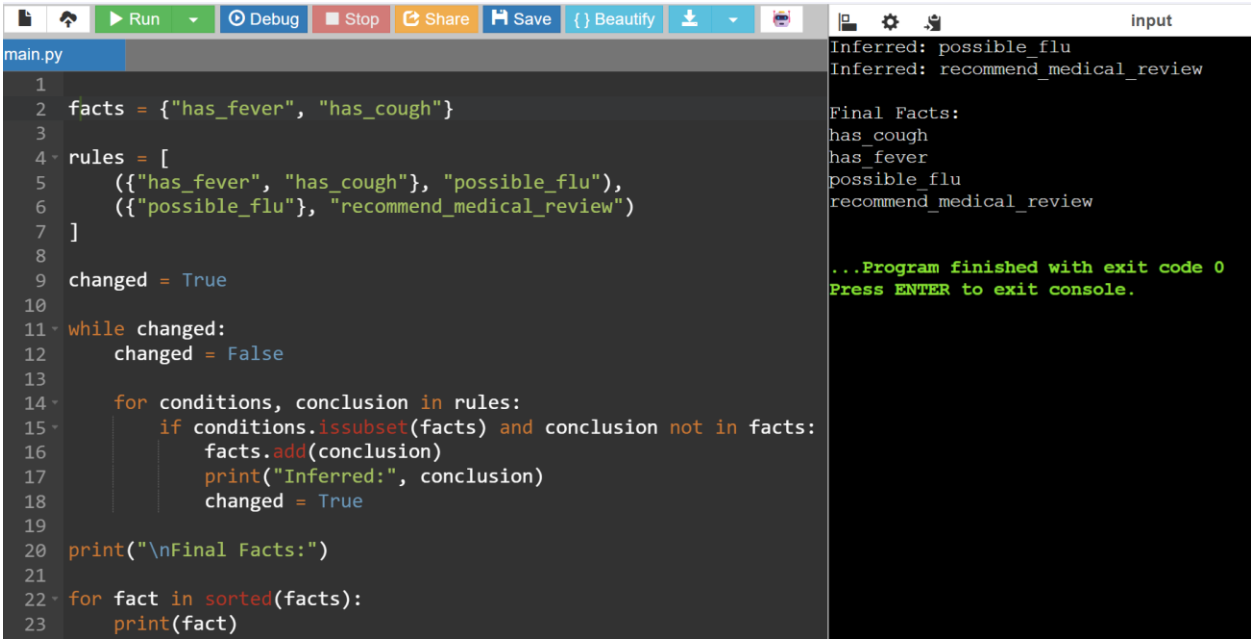
```
Semantic Network Relationships:
Dog -- is-a --> Mammal
Dog -- has-instance --> Tommy
Mammal -- is-a --> Animal

...Program finished with exit code 0
Press ENTER to exit console.
```

Experiment 6: Frame-Based Knowledge Representation



7. Production Rules and Forward Chaining



8. Ontology Development Using Protégé

HelloWorld.pl

```
% Classes
class(person).
class(student).
class(faculty).
class(course).

% Individuals
instance(arun, student).
instance(pythonProgramming, course).

% Object Property
enrolledIn(arun, pythonProgramming).

% Initial goal
:- initialization(main).

main :-
    write('Classes:'), nl,
    forall(class(X), (write(X), nl)),

    nl,
    write('Individuals:'), nl,
    forall(instance(X, Y),
        (write(X), write(' is a '), write(Y), nl)),

    nl,
    write('Object Property:'), nl,
    forall(enrolledIn(X, Y),
        (write(X), write(' enrolledIn '), write(Y), nl)),

    halt.
```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
Classes:
person
student
faculty
course

Individuals:
arun is a student
pythonProgramming is a course

Object Property:
arun enrolledIn pythonProgramming
```

9. Knowledge Graph Construction Using Python

main.py

```
import networkx as nx

knowledge_graph = nx.DiGraph()

knowledge_graph.add_edge("Arun", "Python", relation="studies")
knowledge_graph.add_edge("Arun", "Computer Science", relation="belo
knowledge_graph.add_edge("Python", "Programming Language", relation

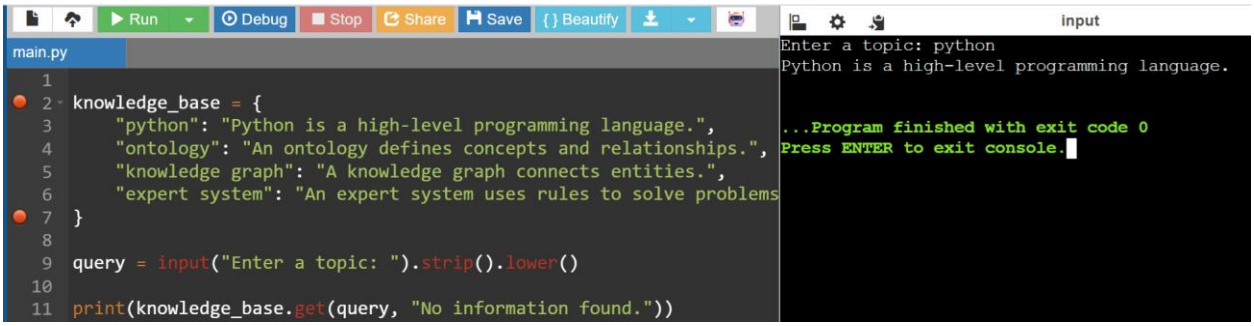
print("Knowledge Graph Triples:")

for subject, obj, data in knowledge_graph.edges(data=True):
    print(subject, "--", data["relation"], "-->", obj)
```

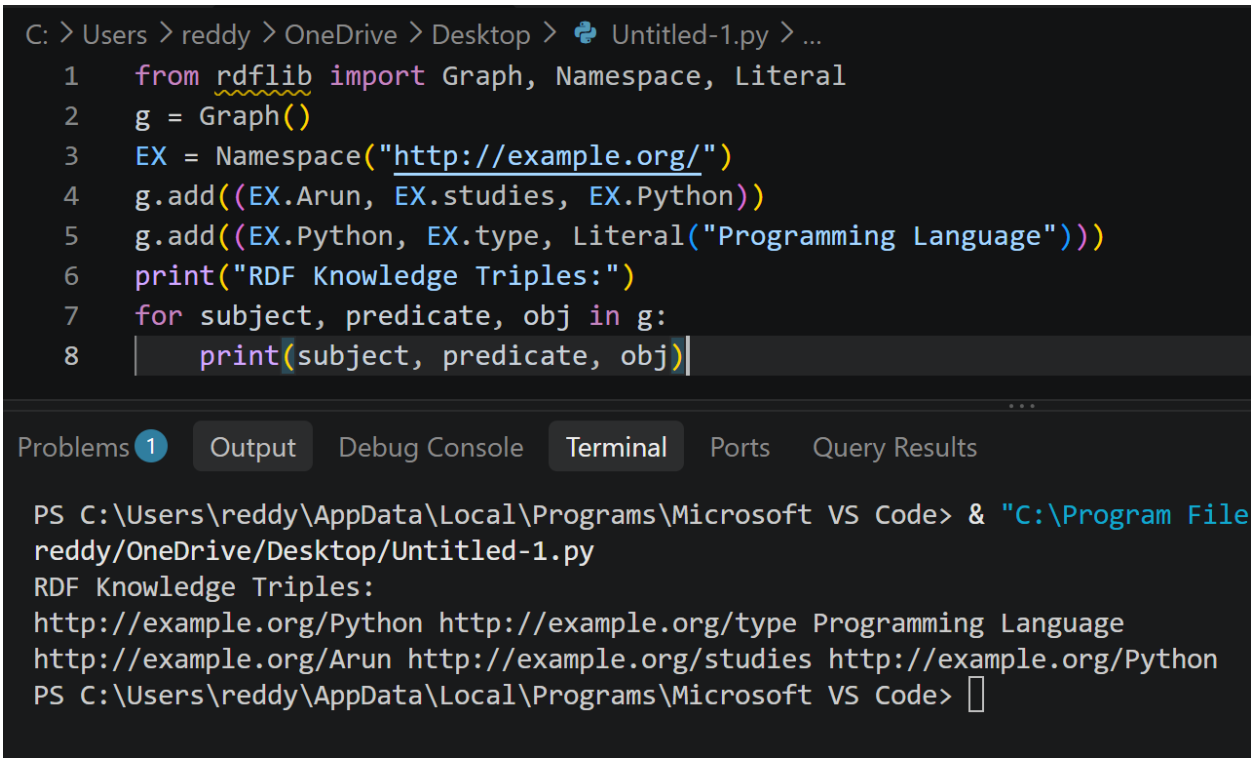
```
Knowledge Graph Triples:
Arun -- studies --> Python
Arun -- belongs_to --> Computer Science
Python -- is_a --> Programming Language


...Program finished with exit code 0
Press ENTER to exit console.
```

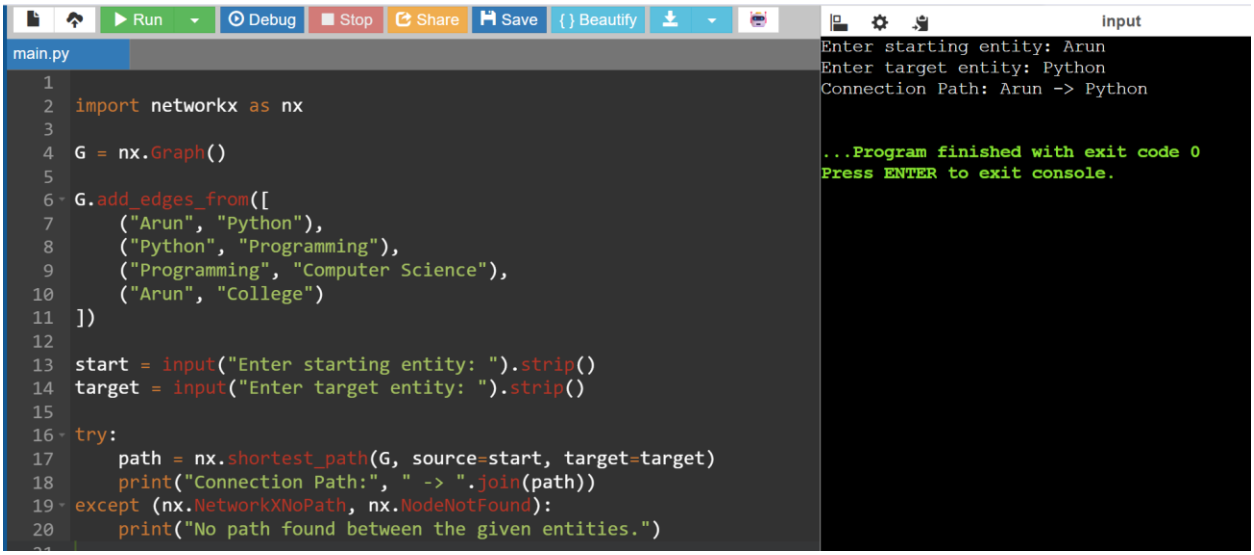
10. Knowledge Management System



11. RDF Knowledge Representation Using RDFLib



12. Knowledge Graph Traversal Using NetworkX


Run
Debug
Stop
Share
Save
{ } Beautify
main.py
import networkx as nx
G = nx.Graph()
G.add_edges_from([
("Arun", "Python"),
("Python", "Programming"),
("Programming", "Computer Science"),
("Arun", "College")
])
start = input("Enter starting entity: ").strip()
target = input("Enter target entity: ").strip()
try:
path = nx.shortest_path(G, source=start, target=target)
print("Connection Path:", " -> ".join(path))
except (nx.NetworkXNoPath, nx.NodeNotFound):
print("No path found between the given entities.")
Input
Enter starting entity: Arun
Enter target entity: Python
Connection Path: Arun -> Python
...Program finished with exit code 0
Press ENTER to exit console.